

Adaptive FIR filter with coefficient update engine

The purpose of this project is to design and implement a digital adaptive finite impulse response (FIR) filter with a sign-sign least-mean-squares (LMS) coefficient update engine, providing a general-purpose adaptive filtering architecture with applications in interference cancellation, including biomedical applications as demonstrated by Widrow et al, in Adaptive Noise Cancelling: Principles and Applications.

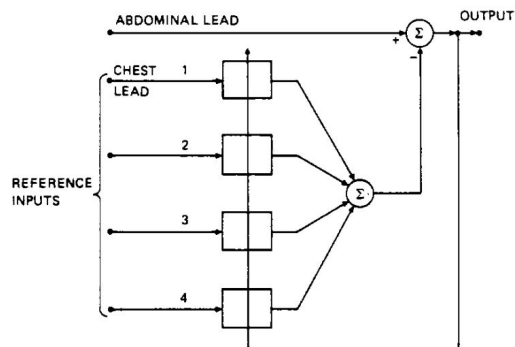
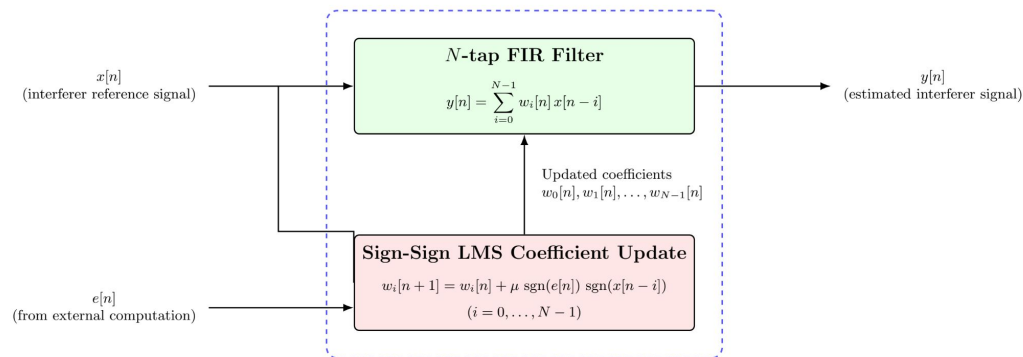
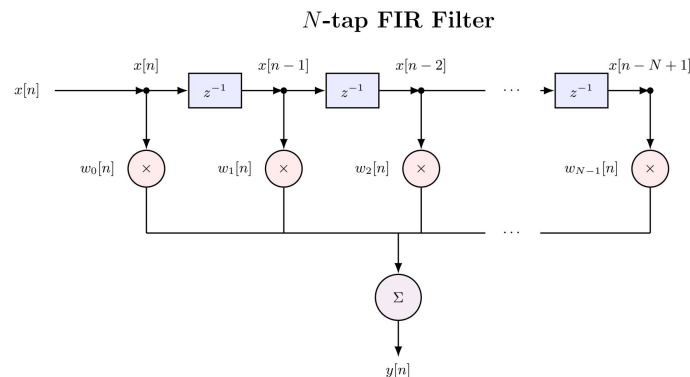


Fig. 15. Multiple-reference noise canceller used in fetal ECG experiment.



I/O Pin Assignment		
Name	Direction	Assignment
clk	Input	Clock input
rst_n	Input	Active low reset
ui_in[7:0]	Input	Time multiplexed input and error
uo_out[7:0]	Output	Corrected signal or debug outputs
uio[7:0]	Bidir	Debug mode: - Output phase tracking - Debug commands (taps, weights, etc.) Inference mode: - Input phase tracking

Design Specifications	
Number of taps (FIR)	8
Bits per tap	8
Output precision	19
Bits per weight	8
Input frequency	1 MHz
Clock rate	Minimum goal: 4 MHz, Stretch goal: 40 MHz

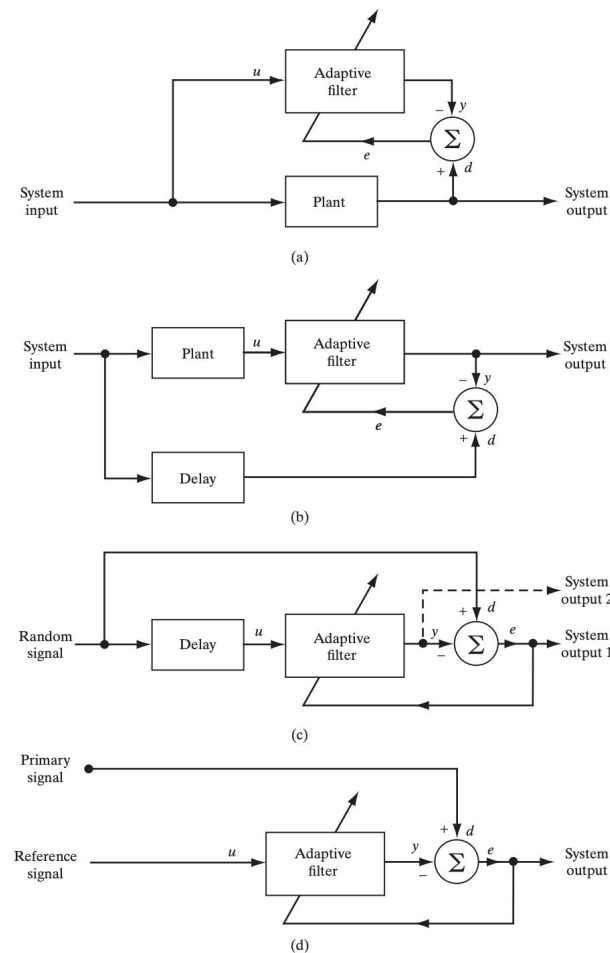


FIGURE 11 Four basic classes of adaptive filtering applications: (a) class I: identification; (b) class II: inverse modeling; (c) class III: prediction; (d) class IV: interference cancellation.

ID	Task Name	2026-09		2026-10						2026-11			
		20	27	04	11	18	25	01	08	15	22		
1	Signal simulation												
2	Repo setup												
3	Design RTL												
4	Test With CocoTB												
5	Implementation + timing verification												
6	Back annotation, synthesis, parasitic extraction												
7	Github Actions												

At a high level, we aim to finish RTL design by Oct 17 (concurrent alongside verification), verification by Oct 17, implementation and timing by Oct 31, back annotation + synthesis + parasitic extraction by Nov 14.

Create Repo

- Add information

Implementation:

- FIR block
- Documentation

Testing

- Timing
- Block-level verification
- Synthesis, parasitic extraction

Implementation:

- Top level module instantiation
- Tiny Tapeout pin connections
- Control FSM

Testing

- Timing
- Output verification
- Synthesis, parasitic extraction

Implementation:

- LMS engine
- Documentation

Testing

- Block-level verification
- Synthesis, parasitic extraction
- MATLAB coefficient simulation